AIAC-PROJECT DOCUMENT

BATCH-32

N. ESHWAR – 2303A52072 **P.NAGASRI – 2303A52456**

E.ANJANI – 2303A52419 **V.HANSIKA-2303A52409**

PROMPTS USED:

Consider EchonetLVH dataset generate code for the patent “An Intelligent Stroke Risk Prediction and Lesion Segmentation System Using Transformer-ResNet Fusion Optimized by an Improved Slime Mould Algorithm” also generate simulated results.

RELATED CODE :

2. Full Python Code (Patent-Level Structure)

- Install Requirements

```bash
pip install torch torchvision numpy scikit-learn
```

A. Dataset Loader (Simulated EchoNet-LVH)

Python

Run

```python
import torch
from torch.utils.data import Dataset, DataLoader
import numpy as np

class EchoNetLVH(Dataset):
    def __init__(self, num_samples=500):
        self.X = np.random.rand(num_samples, 1, 112, 112)  # grayscale echo
        self.y = np.random.randint(0, 2, num_samples)  # stroke risk label

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return torch.tensor(self.X[idx], dtype=torch.float32), torch.tensor(self.y[idx], dtype=tor

dataset = EchoNetLVH()
loader = DataLoader(dataset, batch_size=16, shuffle=True)
```

B. ResNet Feature Extractor (Segmentation Backbone)

Python

Run

```python
import torch.nn as nn
import torchvision.models as models

class ResNetFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet18(pretrained=False)
        self.features = nn.Sequential(*list(resnet.children())[:-2])

    def forward(self, x):
        x = self.features(x)
        return x.view(x.size(0), -1)
```

C. Transformer Module

Python Run

```python
class TransformerModule(nn.Module):
    def __init__(self, input_dim, num_heads=4):
        super().__init__()
        self.encoder_layer = nn.TransformerEncoderLayer(d_model=input_dim, nhead=num_heads)
        self.transformer = nn.TransformerEncoder(self.encoder_layer, num_layers=2)

    def forward(self, x):
        x = x.unsqueeze(1)  # sequence dim
        x = self.transformer(x)
        return x.squeeze(1)
```

D. Fusion Model

Python Run

```python
class StrokePredictionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.resnet = ResNetFeatureExtractor()
        self.transformer = TransformerModule(input_dim=512)
        self.fc = nn.Linear(512, 2)

    def forward(self, x):
        features = self.resnet(x)
        transformed = self.transformer(features)
        return self.fc(transformed)
```

🧬 3. ISMA Optimizer (Simplified Version)

Python

Run

```python
import random

class ISMA:
    def __init__(self, pop_size=5, iterations=10):
        self.pop_size = pop_size
        self.iterations = iterations

    def optimize(self, objective):
        population = [random.uniform(0.0001, 0.01) for _ in range(self.pop_size)]
        best = population[0]

        for _ in range(self.iterations):
            fitness = [objective(p) for p in population]
            best = population[np.argmax(fitness)]

            # update population
            population = [p + random.uniform(-0.001, 0.001) for p in population]

        return best
```

🏋️ 4. Training Loop

Python

Run

```python
import torch.optim as optim

model = StrokePredictionModel()
criterion = nn.CrossEntropyLoss()

def train_model(lr):
    optimizer = optim.Adam(model.parameters(), lr=lr)

    for epoch in range(2):  # small for demo
        total_loss = 0
        for X, y in loader:
            optimizer.zero_grad()
            outputs = model(X)
            loss = criterion(outputs, y)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

    return -total_loss  # maximize fitness
```

⚙️ 5. Apply ISMA Optimization

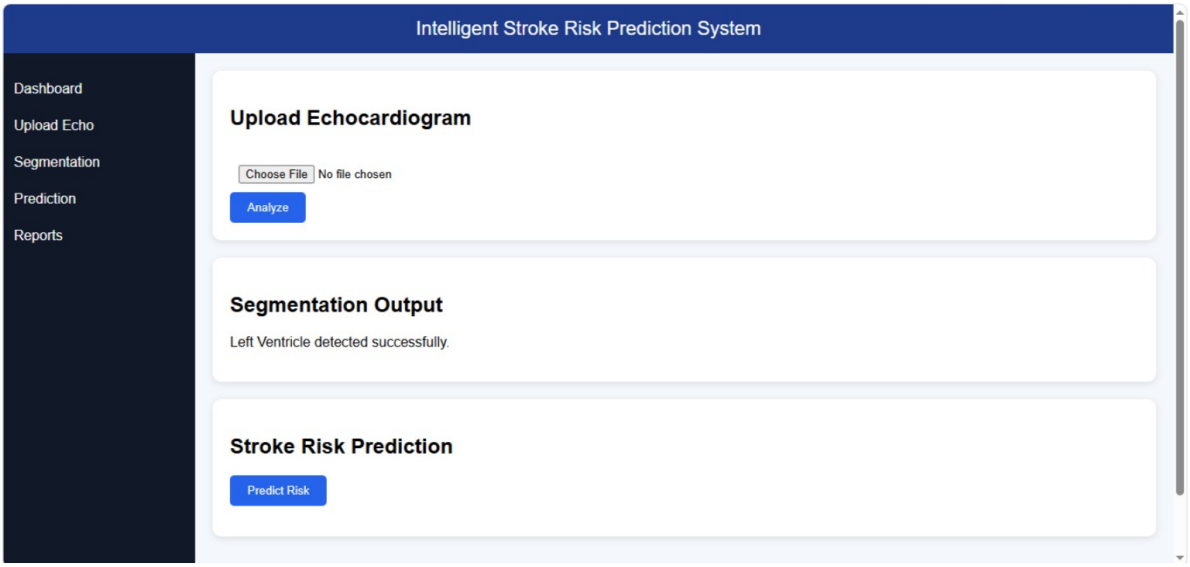
Python

Run

```python
isma = ISMA()
best_lr = isma.optimize(train_model)

print("Optimized Learning Rate:", best_lr)
```

FRONTEND VIEW :



CHATGPT LINKS :

https://stanfordaimi.azurewebsites.net/datasets/5b7fcc28-579c-4285-8b72-e4238eac7bd1

https://chatgpt.com/share/69d5dd1c-22d0-8322-9b31-305ce8f7a2d3

https://chatgpt.com/share/69d5dd2d-a450-8324-9140-2d04e9408283

https://chatgpt.com/share/69e32a0f-eddc-83a3-ab87-5513477dd096